

O'REILLY®



System Design on AWS

Building and Scaling Enterprise Solutions

Jayanth Kumar & Mandeep Singh
Foreword by Swami Sivasubramanian

System Design on AWS

Building and Scaling Enterprise Solutions

Jayanth Kumar and Mandeep Singh
Foreword by Swami Sivasubramanian

System Design on AWS

by Jayanth Kumar and Mandeep Singh

Copyright © 2025 Jayanth Kumar and Mandeep Singh. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Megan Laddusaw

Development Editor: Melissa Potter

Production Editor: Katherine Tozer

Copyeditor: Shannon Turlington

Proofreader: Piper Content Partners

Indexer: nSight, Inc.

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

February 2025: First Edition

Revision History for the First Edition

- 2025-02-14: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098146894> for release details.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *System Design on AWS*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-14689-4

[LSI]

Foreword

Systems design is one of the most important aspects of building apps that are scalable, available, performant, and secure, even when your usage increases by an order of magnitude or two. That's why I found the topic of this book to be so important. I like that this book starts with covering the basics of trade-offs in distributed systems, such as the CAP theorem, units of blast radius, and other common trade-offs in systems design, and then moves to how to apply these principles in practice. I found this approach—going from core principles to actually putting them into practice on critical design decisions (such as “What is the best database for running my workload?” and “What's the best caching strategy for my workloads?” and “Which storage service should I choose?”)—to be especially helpful as it teaches people how to think about these problems even as new capabilities from AWS emerge, making the concepts in this book more durable.

Personally, reading this book reminded me so much of the early days of AWS, when we spent a lot of time discussing these trade-offs. At that time, concepts like CAP theorem, blast radius, and seamless scalability were not well understood, and we had robust discussions around these topics and the inherent trade-offs they come with. However, those discussions led to some incredibly robust AWS services, such as S3, DynamoDB, and many others that millions of AWS customers love. I hope the you grasp these concepts and are inspired to build similarly amazing products and services that delight their customers.

Congratulations to the authors for writing such an excellent book, and best of luck to all who embark on this journey!

Swami Sivasubramanian,
Vice President, AI and Data at AWS

Preface

System design is a disciplined art, so let's create some large-scale masterpieces.

System Design on AWS is a comprehensive guide to helping software architects and engineers understand, build, and scale complex systems on the Amazon Web Services (AWS) cloud. The book provides a structured approach to tackling real-world system design challenges on AWS in a continuously evolving cloud landscape. The material focuses on fundamental principles, architectural trade-offs, and practical use cases, guiding you from Day 0 (minimum viable product) to Day N (most valuable product). Through detailed chapters, this book demystifies AWS's wide-ranging services and how they can be strategically leveraged to address both technical and business requirements.

How This Book Is Organized

Organized into three core parts, this book begins by establishing a foundation in system design, then dives into the essential AWS services for building resilient architectures, and concludes with practical system design use cases. Foundational system design patterns, scaling mechanisms, caching strategies, network protocols, and deployment strategies are covered. By exploring these elements in a cloud context, the book offers a toolkit for approaching and solving large-scale design problems in any industry. Each part opener breaks down the chapters included in that part, so take a look at those openers for details. This book aims to equip you with:

- A solid understanding of fundamental system design principles and trade-offs
- Proficiency in key AWS services and their roles in building large-scale systems
- Insight into real-world use cases that illustrate how to structure and deploy scalable architectures on AWS
- Strategies to balance performance, scalability, and cost in AWS cloud environments

What This Book Isn't

This book does not provide detailed code implementations for deploying the infrastructure or application business logic. Instead, it emphasizes the architecture and design process, helping you understand how to break a system down into manageable components and make informed architectural decisions.

Who Should Read This Book

This book is ideal for software architects, developers, DevOps engineers, and system architects who want to build or scale applications on AWS. It is especially useful for professionals transitioning from on-premises to cloud infrastructure or aiming to leverage AWS for large-scale applications. A basic understanding of software implementation and familiarity with one programming language are recommended.

This book is also recommended for people holding leadership positions in software development and DevOps. The guidance and principles in the book will help you remove team blockers while building systems and making informed decisions.

While this book is titled *System Design on AWS* and the focus is on the AWS cloud, we recommend it to everyone who loves to build systems on any of the cloud providers' platforms or for on-premises data centers as well as anyone who is just curious about building systems. The design and scaling principles discussed in the book are agnostic of cloud providers.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

NOTE

This element signifies a general note.

Many of the tools and services discussed in these pages are abbreviated on repeated mentions for your convenience. For example, *Amazon Elastic Compute Cloud* is sometimes called *Amazon EC2* or just *EC2*.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/SystemDesignOnAWS>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

This book is the culmination of insights, experience, and feedback from numerous collaborators and industry experts. We thank our colleagues, reviewers, and readers for their contributions and dedication to advancing cloud-based system design.

We extend our sincere gratitude to the entire O'Reilly team, both visible and behind the scenes, for their contributions and feedback on this book. We would not have met with the O'Reilly team without an introduction from John Culkin and his expert review of our early proposal of this book. We would like to thank Megan Laddusaw for support on the book proposal and getting the chapters outline ready. A special thanks to Melissa Potter, our development editor, for her patience and support. Her initial reviews were incredibly helpful in structuring the content and helped make the book more valuable.

A big thanks to Swami Sivasubramanian for taking the time to write a wonderful foreword; his words perfectly set the stage, and we couldn't have asked for a better introduction. Thank you to Harshit Bangar, Rakesh Kumar, Neha Shetty, Dinesh Chittibala, Alankrit Kharbanda, Kristina Manukyan, and Rohn Arya for reviewing the book and providing many helpful suggestions and insightful comments.

We would also like to thank the Dragonfly DB team for sponsoring the two early-release chapters on their website and making them freely available for the tech community. Special thanks to all the companies out there publishing detailed engineering blogs and sharing the innovations and deep dives.

Mandeep Singh

I would like to thank my wonderful wife, Priyanka, for her patience and encouragement and for putting up with me spending long hours writing. I am forever grateful to my parents for enabling me to achieve my dreams and always putting my interests before theirs. Thanks to all my family members for their support and guidance.

Thanks to my college, Army Institute of Technology, for introducing me to computer science and helping me become a disciplined individual. Thanks to all the amazing opportunities I had working at different organizations: AWS, Amazon, Jupiter, and Skytap (a Kyndryl company). I met a lot of amazing folks, and they have greatly shaped my thinking around decision making and systems building. I would also like to thank my friends, teachers, and professional connections for suggestions and discussions along the way.

The idea of writing a book would never have originated without my coauthor, Jayanth. Thanks for sharing the opportunity with me—the entire writing of this book has been entirely a new experience and learning for me.

Jayanth Kumar

This book is a heartfelt expression of gratitude to my family and all the teachers who have guided me throughout my life. I am forever grateful for their support, encouragement, and wisdom that have

profoundly shaped who I am. I got the inspiration to author such a technical book from my professors, **Pushpak Bhattacharya** and **George Varghese**, who infected me with their enthusiasm for the music of systems and algorithmics.

I owe much of my journey to the institutions that nurtured my curiosity and skills: my alma maters IIT Bombay and the University of California, Los Angeles. These environments were instrumental in developing my foundation and pushing me to explore new horizons in technology and system design.

My career has been a canvas of experiences, shaped by incredible mentors and colleagues at SAP, Goodhealthapp, Delhivery, Amazon, and Turl Street Group. Each role allowed me to build and scale systems through various growth stages, from 0 to 1 at Goodhealthapp (startup), 1 to 10 at Delhivery (unicorn), 10 to 1,000 at SAP (enterprise), and 1,000+ at Amazon (FAANG), and each taught me invaluable lessons in resilience, adaptability, and innovation.

This book could not have come to life without the dedication of my coauthor, Mandeep Singh, who tirelessly worked alongside me, often burning the midnight oil to cover ground where I left off. His commitment and attention to detail have been invaluable in bringing this work to completion.

There are a lot of folks whom I wish I could mention individually, whose contributions, guidance, and shared knowledge shaped this book, but I am constrained by space, so please know that your impact is felt and appreciated. I am also indebted to the broader tech community, whose inspiration fuels my dedication to this field. It is my hope that this book will add value to our shared pursuit of innovation and learning.

Part I. System Design Basics

Perfection is finally attained not when there is no longer anything to add but when there is no longer anything to take away.

—Antoine de Saint-Exupéry

The first part of this book will give you a solid foundation in system design, offering insights into the fundamental principles that underpin all types of software systems, ranging from trade-offs to the criteria for picking the right tools and technologies and thus, the right architecture patterns.

In Part I, you will:

- Grasp the system trade-offs inherent in large-scale distributed systems and know how to balance conflicting demands like scalability, reliability, and maintainability
- Develop a clear understanding of options for data storage, including file, block, and object stores, alongside relational and nonrelational databases and methods for scaling them

- Master techniques to achieve low latency and high throughput in storage and compute through caching and content delivery networks (CDNs)
- Learn to horizontally scale systems with load balancers, API gateways, and reverse proxies
- Navigate the choices among communication and network protocols at different layers of the Open Systems Interconnection (OSI) and TCP/IP models and know how to choose when to use what
- Familiarize yourself with the system design architecture patterns that are commonly used in the industry and learn how to adapt them to various system requirements
- Gain insight into the AWS services that map to different system design paradigms, which we will explore further in later chapters

Chapter 1 will delve into the world of trade-offs in large-scale distributed systems. We'll explore essential concepts like reliability, scalability, and maintainability, shedding light on why these trade-offs emerge. We'll tackle common misconceptions through the fallacies of distributed computing and address pivotal choices such as space versus time, latency versus throughput, performance versus scalability, and consistency versus availability. We'll also offer practical guidelines and strategies that have emerged from years of system design wisdom, providing tangible approaches to enhance system performance and efficiency.

Chapter 2 will introduce you to different types of data storage, including file, block, and object stores. We'll cover relational databases, their concepts, and their architecture in this chapter. You'll gain insights into the intricacies of scaling and optimizing relational databases, including techniques like partitioning, indexing, replication, federation, sharding, and denormalization.

Chapter 3 will unveil the architecture of nonrelational databases, such as key-value, wide-column, document, and graph databases, and discuss how leaderless architectures enable scalability through mechanisms like quorum-based replication, consistent hashing, and eventual consistency.

Chapter 4 will introduce you to caching, where we will explore diverse caching strategies spanning different tiers, from client and server to application and database levels. We'll dissect core caching approaches like write-through, read-through, refresh-ahead, write-back, and cache-aside. Moreover, we'll unravel the potential of both push and pull CDNs in reducing latency and improving the efficiency of content delivery.

Chapter 5 will explore load balancers, API gateways, and reverse proxies, and includes discussions on Layer 4 (L4) network scaling and Layer 7 (L7) application layer scaling. Concepts like sticky sessions, forward proxies, and reverse proxies will come to life as we delve into the nuances of creating scalable, resilient architectures.

Chapter 6 will demystify network protocols, unraveling the intricacies of TCP/IP and User Datagram Protocol (UDP), as well as HTTP, SMTP, Extensible Messaging and Presence Protocol (XMPP), and

their suitable applications. You'll gain a comprehensive understanding of synchronous and asynchronous service design and delve into standards like REST, WebRTC, and GraphQL. Mastering these communication concepts will better equip you to make informed choices for different use cases.

Chapter 7 is all about containerization: the creation, management, and orchestration of Docker containers. You'll also be introduced to modern strategies for container deployment, enabling you to manage the complexity of modern cloud native applications.

Chapter 8 will cover different architectural designs and patterns, starting with choreography and orchestration in detail, where we explore asynchronous architectures, message brokers, message queues, and publisher-subscriber patterns. As you delve deeper into large-scale software engineering design, you'll compare microservices with monolithic designs, serverless architecture, the saga pattern, the event-sourcing pattern, big data architectures, and domain-driven design.

After this comprehensive journey through the nuances of system design in Part I, you'll be well prepared to wield these principles and technologies in creating effective and efficient large-scale systems. **Part II** will cover the AWS services in detail, bringing these concepts to light in their use and trade-offs in cloud architecture.

Chapter 1. System Design Trade-offs and Guidelines

Today's modern technological revolution is happening because of large-scale software systems. Big enterprise companies like Google, Amazon, Oracle, and SAP have all built large-scale software systems to run their (and their customers') businesses. Building and operating such large-scale software systems requires first-principles thinking to design and develop the technical architecture before actually putting the system into code. This is because we don't want to be in a state where these systems will not work or scale as more users need them.

If the design is right in the first place, the rest of the implementation journey becomes smooth. This requires looking at the business requirements, understanding the needs and objectives of the customer, evaluating different trade-offs, thinking about error handling and edge cases, and contemplating future changes and robustness while worrying about basic details like algorithms and data structures. Enterprises can avoid the mistake of wasted software-development effort by carefully thinking about systems and investing time in understanding bottlenecks, system requirements, users being targeted, user access patterns, and many such decisions, which, in short, is *system design*.

This chapter covers the basics of system design with the goal of helping you understand the fundamental concepts, the trade-offs that naturally arise in such large-scale software systems, the fallacies to avoid in building such systems, and the guidelines of system design—those lessons that have been learned from building large-scale software systems over the years. This chapter is intended

to introduce you to the essentials—we'll dig into the details in later chapters, but we want you to have a good foundation to start. Let's begin with the basic concepts of system design.

System Design Concepts

To understand the building blocks of system design, we should understand the fundamental concepts around systems. We can leverage *abstraction* here: the concept in computer science of obfuscating the inner details to create a model of these system design concepts, which can help us grasp the bigger picture. The concepts in system design, be it any software system, revolve around communication, consistency, availability, reliability, scalability, system maintainability, and fault tolerance. We will go over each of these concepts in detail, creating a mental model while also exploring how they are applied in large-scale system design.

Communication

A large-scale software system is composed of small subsystems, known as *servers*, that communicate with one another—that is, they exchange information or data over the network to solve a business problem, provide business logic, and compose functionality. Communication can take place in either a synchronous or an asynchronous fashion, depending on the needs and requirements of the system.

Figure 1-1 shows the differences in the sequence of actions in synchronous and asynchronous communication.

NOTE

We will cover the communication protocols as well as mechanisms for asynchronous communication in detail in **Chapter 6**.

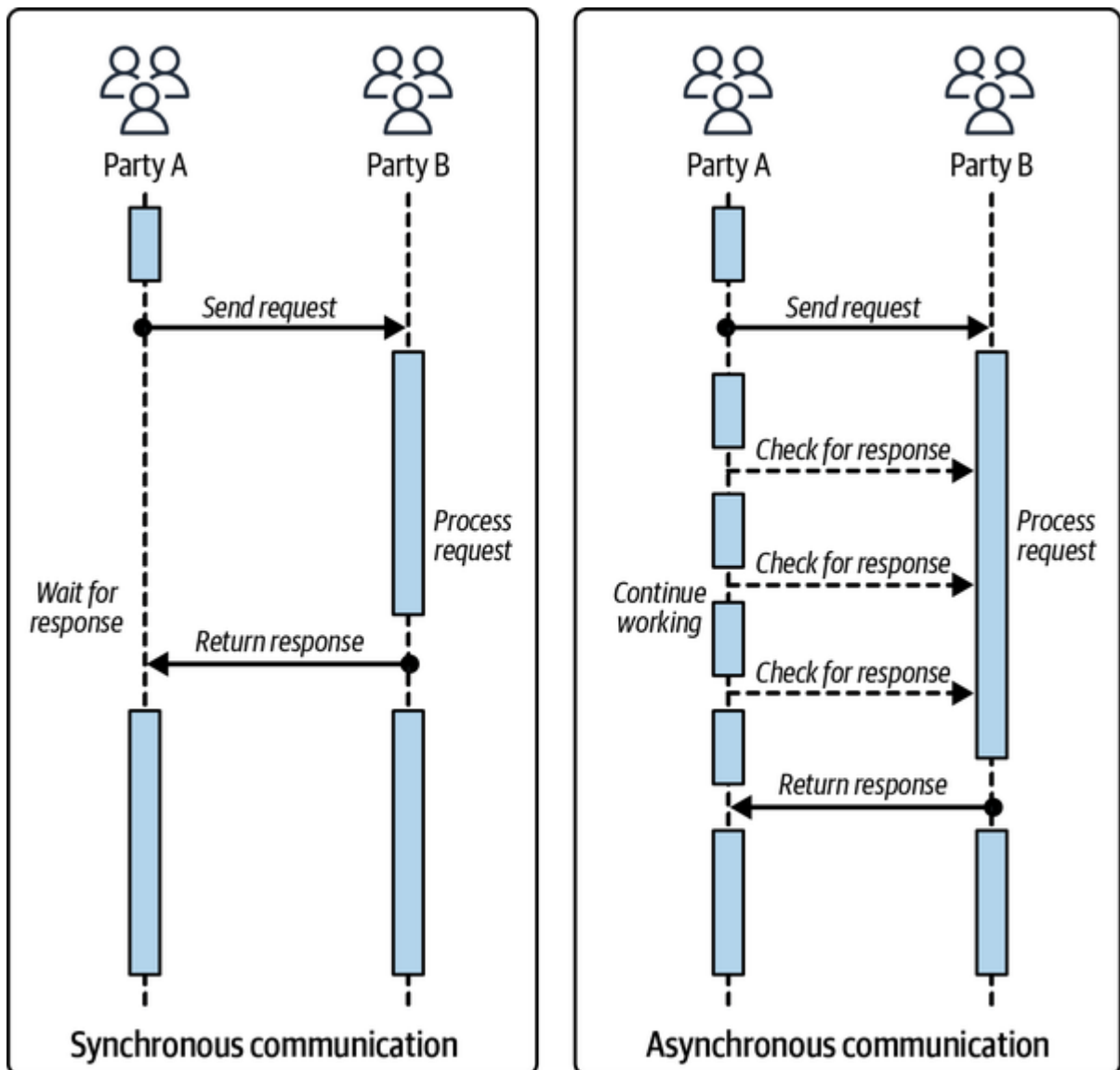


Figure I-1. Sequence diagram for synchronous versus asynchronous communication

Synchronous communication

Imagine having a phone conversation with your friend: you hear them and speak with them at the same time and use pauses in between to allow the conversation to complete. This is an example of *synchronous communication*: a type of communication in which two or more parties communicate with one another in real time. This type of communication is more immediate and allows for quicker resolution of issues or questions.

In system design, a communication mechanism is synchronous when the sender will block (or wait) for the call or execution to return before continuing. This means that until a response is returned by the receiver, the application will not execute any further, which could be perceived by the user as latency or performance lag in the application.

Asynchronous communication

Now, imagine that instead of a phone conversation, you switch to email. As you communicate over email with your friend, you send the message and wait for the reply to come at a later time (but within an acceptable time limit). You also follow a practice to follow up if there is no response after this time limit has passed. This is an example of *asynchronous communication*: a type of communication in which two or more parties do not communicate with one another in real time. Asynchronous communication can also take place through messaging platforms, forums, and social media, where users can post messages and responses may not be immediate.

In system design, a communication mechanism is asynchronous when the sender does not block (or wait) for the call or execution to return from the receiver. Execution continues in your program or system, and when the call returns from the receiving server, a *callback* function is executed. Asynchronous communication is often used in system design when an immediate response is not required or when the system needs to be more flexible and tolerant of delays or failures.

In general, the choice of synchronous or asynchronous communication in system design depends on the specific requirements and constraints of the system. Synchronous communication is often preferred when real-time response is needed (such as the communication between the frontend UI and the backend), while asynchronous communication is often preferred when flexibility and robustness are more important (such as checking the status of a long-running job).

Consistency

Consistency—being consistent or in accordance with a set of rules or standards—is an important requirement when it comes to communication between servers in a software system. Consistency can refer to a variety of concepts and contexts in system design.

In the context of distributed systems, consistency can be the property of all *replica* nodes (more on this in a moment) or servers having the same view of data at a given point in time. This means that all replica nodes have the same data, and any updates to the data are immediately reflected on all replica nodes.

In the context of data storage and retrieval, consistency refers to the property of each read request returning the value of the most recent write. This means that if a write operation updates the value of a piece of data, any subsequent read requests for that data should return the updated value. Let's discuss each of these in more detail.

Consistency in distributed systems

Distributed systems are software systems that are separated physically but connected over the network to achieve common goals using shared computing resources.

Ensuring consistency—that is, providing the same view of the data to each server in a distributed system—can be challenging because multiple replica servers may be in different physical locations and may be subject to different failures or delays.

To address these challenges and ensure data consistency in distributed systems, we can use various techniques:

Data replication

In this approach, multiple copies of the data are maintained on different replica nodes, and updates to the data are made on all replica nodes simultaneously through blocking synchronous communication. This ensures that all replica nodes have the same view of the data at any given time.

Consensus protocols

Consensus protocols ensure that all replica nodes agree on the updates to be made to the data. They can use a variety of mechanisms, such as voting or leader election, to make sure all replica nodes are in agreement before updating the data.

Conflict resolution

If two or more replica nodes try to update the same data simultaneously, conflict resolution algorithms are used to determine which update should be applied. These algorithms can employ various strategies, such as last writer wins or merge algorithms, to resolve conflicts.

Consistency in data storage and retrieval

Large-scale software systems produce and consume a large amount of data, and thus, ensuring consistency in data storage and retrieval is important for maintaining the accuracy and integrity of the data in these systems. For example, consider a database that stores the balance of a bank account. If we withdraw money from the account, the database should reflect the updated balance immediately. If the database does not ensure consistency, it is possible for a read request to return an old balance, which could lead to incorrect financial decisions or even financial losses for us or our banks.

To address these challenges and ensure read consistency in data storage systems, we can use various techniques:

Write-ahead logging

In this technique, writes to the data are first recorded in a log before they are applied to the actual data. This ensures that if the system crashes or fails, the data can be restored to a consistent state by replaying the log.

Locking

Locking mechanisms are used to ensure that only one write operation can be performed at a time. This ensures that multiple writes do not interfere with one another and that reads always return the value of the most recent write.

Data versioning

In this technique, each write operation is assigned a version number, and reads always return the value of the most recent version. This allows for multiple writes to be performed concurrently while still ensuring that reads return the value of the most recent write.

NOTE

We will discuss some of these techniques for ensuring consistency in detail in Chapters 2 and 3.

Consistency spectrum model

Since consistency can mean different things, the *consistency spectrum model* helps us reason about whether a distributed system is working correctly when it's doing multiple things at the same time, like reading, writing, and updating data. The consistency spectrum model represents the various consistency guarantees that a distributed system can offer, ranging from eventual consistency to strong consistency. The consistency guarantee chosen depends on the specific requirements and constraints of the system. Let's walk through the consistency levels in the consistency spectrum model:

Strong consistency

At one end of the spectrum, strong consistency guarantees that all replica nodes have the same view of the data at all times and that any updates to the data are immediately reflected on all replica nodes. This ensures that the data is always accurate and up to date, but it can be difficult to achieve in practice as it requires all replica nodes to be in constant communication with one another.

NOTE

We will cover the strong consistency requirements of relational databases as part of the ACID model in Chapter 2.

Monotonic read consistency

Monotonic read consistency guarantees that once a client has read a value from a replica node, all subsequent reads from that client will return the same value or a more recent value. This means that a client will not see “stale” data that has been updated by another client. This provides a stronger consistency guarantee than eventual consistency because it ensures that a client will not see outdated data.

Monotonic write consistency

Monotonic write consistency guarantees that once a write operation has been acknowledged by a replica node, all subsequent reads from that replica node will return the updated value. This means that a replica node will not return outdated data to clients after a write operation has been acknowledged. This provides a stronger consistency guarantee than eventual consistency because it ensures that a replica node will not return outdated data to clients.

Causal consistency

Causal consistency works by categorizing operations into dependent and independent operations. Dependent operations are also called *causally related operations*. Causal consistency preserves the order of the causally related operations. It guarantees that if two operations are causally related in dependent and independent operations, then they will be seen in the same order by all processes in the system. This means that if operation A must happen before operation B, then all processes in the system will see A before they see B. This provides a stronger consistency guarantee than eventual consistency because it ensures that the order of related operations is preserved.

Eventual consistency

At the other end of the spectrum, eventual consistency guarantees that, given enough time, all replica nodes will eventually have the same view of the data. This allows for more flexibility and tolerance of delays or failures but can result in temporary inconsistencies in the data.

NOTE

We will cover the tunable consistency feature of some of the nonrelational columnar databases like Cassandra in **Chapter 3**.

Figure 1-2 shows the difference in the results of performing a sequence of actions under either strong consistency or eventual consistency. As you can see on the left showing strong consistency, when x is read from a replica node after updating it from 0 to 2, it will block the request until replication happens and then return 2 as the result. On the right illustrating eventual consistency, you can see that when the replica node is queried, it will give a stale result of x as 0 before replication completes.

The consistency spectrum model therefore provides a framework for understanding the trade-offs between consistency and availability in distributed systems and helps system designers choose the appropriate consistency guarantee for their specific needs.

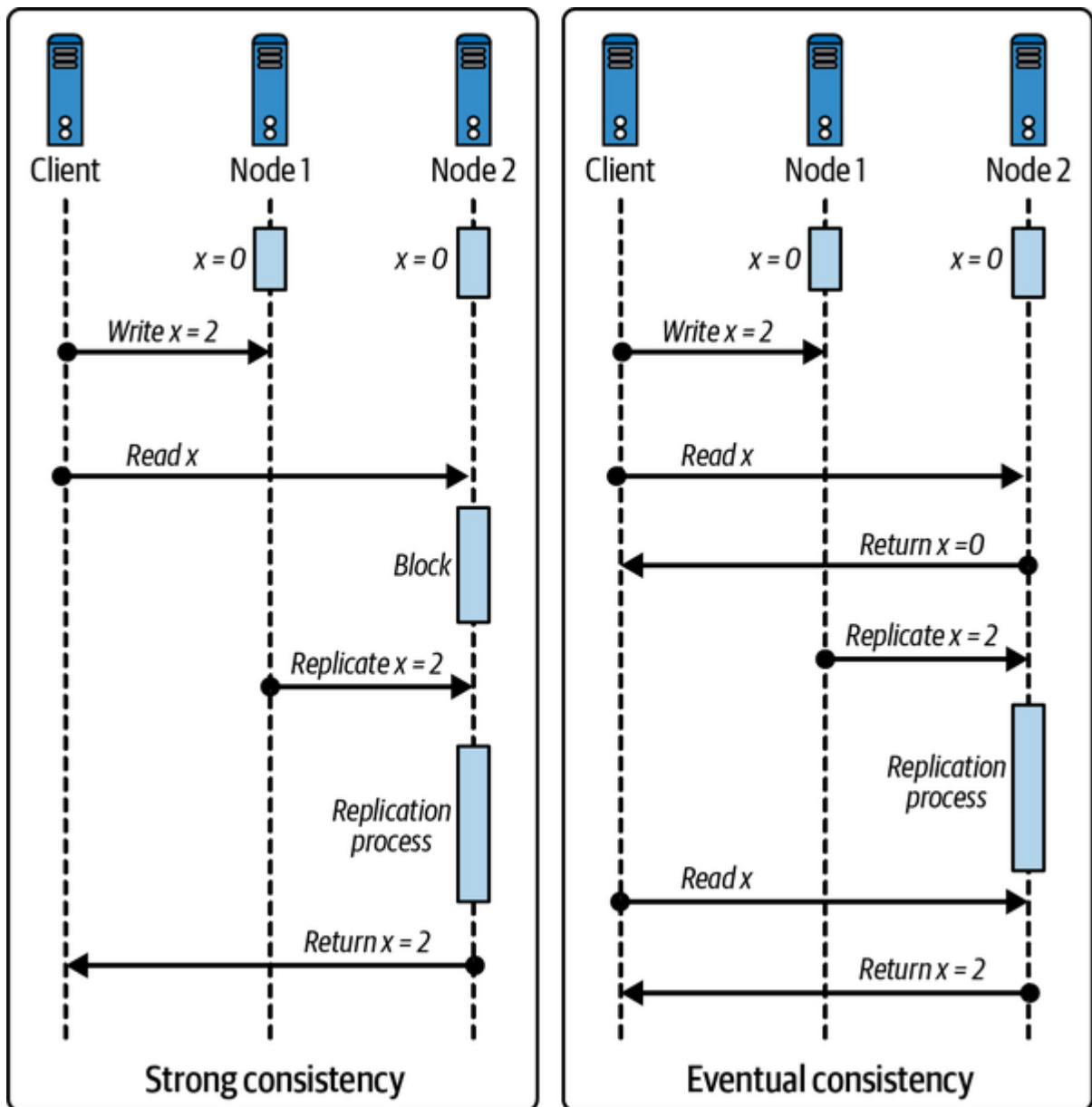


Figure 1-2. Sequence diagrams for strong consistency and eventual consistency

The level of consistency that a distributed system aims to achieve directly affects its complexity: stricter consistency models (like strong consistency) require extensive coordination, synchronization, and consensus across nodes, increasing latency and operational costs and making failure handling more challenging. This complexity is especially pronounced in geographically distributed systems, where network latency and scalability constraints complicate synchronization of global data. Conversely, relaxing consistency requirements (as in eventual consistency) reduces system complexity, enabling higher scalability and availability at the cost of allowing temporary data inconsistencies. Ultimately, there's a trade-off between data accuracy and system simplicity, with each choice tailored to the specific needs and tolerance for latency, availability, and correctness in the application.

Availability

In a large-scale software system, subsystems or servers can go down and may not be fully available to respond to the client's requests; this is referred to as the system's *availability*. A system that is highly available can process requests and return responses in a timely manner, even under heavy load or when there are failures or errors. Let's try to quantify the measurement of availability of the system.

Measuring availability

Availability can be measured mathematically as the percentage of the time the system was up (total time – time system was down) over the total time the system should have been running:

$$\text{Availability\%} = \frac{(\text{Total Time} - \text{Sum total of time system was down})}{\text{TotalTime}} \times 100$$

Availability percentages are represented in nines based on this formula over a period of time. You can see a breakdown of what these numbers really work out to in [Table 1-1](#).

Table 1-1. Availability percentages represented in nines

Availability percentage	Downtime per year	Downtime per month	Downtime per week
90% (1 nine)	36.5 days	72 hours	16.8 hours

This document was truncated here because it was created in the Evaluation Mode.